

Use Case 3: Pipeline Leak Detection & Integrity Agent

Segment: Midstream | Claude Code + Bedrock AgentCore Hackathon · Energy Symposium

The Problem

A midstream operator manages 200 miles of natural gas transmission pipeline with 8 SCADA measurement stations. Leaks manifest as pressure drops and flow imbalances between adjacent stations, but distinguishing a real leak from normal operational transients (compressor starts, valve changes, temperature-driven line pack shifts) requires contextual reasoning that rule-based SCADA alarms cannot do.

The result is a lose-lose situation: set the alarm threshold too sensitive and you get constant false alarms that cause unnecessary shutdowns (\$100K+ per event) and alarm fatigue. Set it too conservative and you miss real leaks. Over 40% of pipeline leaks in remote areas go undetected for more than a week, a delay that can escalate repair costs by 600% while risking environmental disasters.

DOT PHMSA penalties for pipeline incidents reach up to \$2.7M per violation. Modern AI-driven leak detection systems achieve 98% accuracy in pinpointing leaks and reduce detection time from days to minutes. The challenge is not just detection, it is intelligent disambiguation: knowing when an anomaly is a real leak vs. a compressor transient vs. a cold morning causing line pack contraction.

Your Task

Solve the problem above using Agentic AI — built with Claude Code and deployed on Amazon Bedrock AgentCore.

Use the dataset and reference documents provided in this package:

- **SCADA history** — `scada_timeseries.csv` (207K readings, 8 stations, 90 days at 5-minute intervals)
- **Labeled ground truth** — `labeled_leak_events.csv` (5 real leaks) and `labeled_false_positive_events.csv` (15 events that look like leaks and aren't)
- **Corroborating context** — `pipeline_segment_metadata.csv`, `gas_composition.csv`, `weather_conditions.csv`, `valve_status.csv`, `row_encroachment.csv`

- **Integrity history** — `inspection_history.csv` (ILI runs), `cathodic_protection.csv`
- **Reference documents** (`data/reference_docs/` , `.md` and `.pdf`) — pipeline operating procedures, DOT/PHMSA regulatory reference (49 CFR 191, Form 7100.1)

What you decide

Everything else. What the application actually does, which question it answers and who it answers it for, how many agents and how they divide the work, which orchestration pattern, what each tool does, which AgentCore modules you use, and what the interface looks like — all of that is your team’s call.

There is deliberately no worked solution and no capability checklist in this document. Deciding what is worth building from the problem and the data is the hackathon. Two teams solving this well should end up with two visibly different applications.

What “grounded” means here

Every number and claim your application states should trace back to a specific file, row, or passage in the data above — and it should be able to say which one. An answer that sounds expert but cites nothing is worth less than a narrower answer that shows its evidence. If the data doesn’t support a conclusion, the right behavior is to say so, not to fill the gap.

Judges score the working demo, the depth of the application’s reasoning, and whether its claims trace back to this data.

Dataset Provided

All files live under `data/` .

Join gotcha, check this before you join anything on `segment_id` . Every file below uses the prefixed form (`SEG-01` , `SEG-02` , ...) — except item 9, `valve_status.csv` , whose `segment_id` is a bare two-digit string (`01` , `02` , ...). Normalize the format before joining `valve_status` to anything else, or the join will silently return zero rows.

1. SCADA Time-Series CSV – `scada_timeseries.csv`

90 days of 5-minute readings for 8 pipeline stations – 207,360 rows, 2025-12-01 00:00 to 2026-02-28 23:55 (25,920 readings per station).

Column	Description
<code>timestamp</code>	ISO 8601 datetime
<code>station_id</code>	ST-01 through ST-08

Column	Description
station_type	compressor / meter / custody_transfer
pressure_psi	Line pressure at station
pressure_upstream_psi	Upstream tap pressure
flow_mmscfd	Volumetric flow rate (million standard cubic feet per day)
flow_direction	inlet / outlet
temperature_f	Gas temperature at station
compressor_status	running / standby / shutdown
compressor_speed_rpm	Shaft speed (0 if not running)
valve_position_pct	Control valve position (0-100%)
line_pack_mmscf	Calculated line pack between this and next station
mass_balance_deficit_mmscfd	Inlet minus outlet flow (positive = potential loss)
event_flag	normal / compressor_start / valve_change / leak / false_positive

2. Labeled Leak Events CSV – `labeled_leak_events.csv`

5 events: `event_id`, `onset_timestamp`, `true_leak_location_mile_marker`, `affected_segment`, `leak_rate_mmscfd`, `severity` (seep / moderate / significant / near_rupture), `detection_lag_minutes`

3. Labeled False Positive Events CSV – `labeled_false_positive_events.csv`

15 events: `event_id`, `timestamp`, `station_id`, `fp_type` (compressor_start / valve_change / temperature_line_pack), `pressure_drop_psi`, `duration_minutes`, `explanation`

4. Pipeline Segment Metadata CSV – `pipeline_segment_metadata.csv`

7 segments (`SEG-01` through `SEG-07`) spanning the 8 stations – 200 miles total, 24-inch, X65, MAOP 850 PSI.

`segment_id`, `from_station`, `to_station`, `length_miles`, `diameter_in`, `wall_thickness_in`, `material_grade`, `max_operating_pressure_psi`, `elevation_start_ft`, `elevation_end_ft`, `valve_locations_mile_markers`, `maop_psi`

5. Gas Composition CSV – [gas_composition.csv](#)

104 rows (per station, per sample date). [date](#), [station_id](#), [methane_pct](#), [ethane_pct](#), [propane_pct](#), [co2_pct](#), [n2_pct](#), [h2s_ppm](#), [heating_value_btu_scf](#), [specific_gravity](#), [compressibility_factor_z](#) – required for accurate mass balance and line pack calculations.

6. Weather Conditions CSV – [weather_conditions.csv](#)

2,160 hourly observations over the 90-day window: [timestamp](#), [ambient_temp_f](#), [precipitation_in](#), [wind_speed_mph](#), [humidity_pct](#), [ground_temp_f](#), [frost_heave_risk](#). Ambient and ground temperature across the same 90-day window. Line pack contracts on a cold morning, which moves pressure and flow in its own right.

7. Inspection History CSV – [inspection_history.csv](#)

80 in-line inspection (ILI) records keyed on [segment_id](#). Known wall-loss anomalies and prior findings per segment.

8. Cathodic Protection CSV – [cathodic_protection.csv](#)

1,890 rows (per segment, per day). CP rectifier and pipe-to-soil potential readings. Failing CP is a leading indicator of external corrosion.

9. Valve Status CSV – [valve_status.csv](#)

1,890 rows (per valve, per day): [valve_id](#), [segment_id](#), [date](#), [position_pct](#), [state](#), [response_time_sec](#), [leak_test_result](#) – per-valve operability. [segment_id](#) format differs from every other file here – see the warning at the top of this section.

10. Right-of-Way Encroachment CSV – [row_encroachment.csv](#)

50 encroachment records keyed on [segment_id](#). Third-party activity near the line – the leading cause of pipeline damage in the US.

11. Operating Procedures – [reference_docs/pipeline_operating_procedures.md](#) / [.pdf](#)

Normal operating envelopes per station, isolation valve procedures, emergency shutdown steps, line pack calculation methodology, pressure transient response guide.

12. DOT PHMSA Regulatory Reference – [reference_docs/dot_phmsa_regulatory_reference.md](#) / [.pdf](#)

49 CFR Part 191 reporting requirements, 5-minute notification rule, NRC contact (1-800-424-8802), required incident report fields, PHMSA Form 7100.1.

Both reference docs are supplied as Markdown and PDF. The Markdown versions are easier to chunk and index; the PDFs are there if you want to demo document parsing.

Logging in, environment setup, and how to submit: see the hackathon portal.

Claude Code: <https://docs.claude.com/en/docs/claude-code> **Strands Agents SDK:** <https://strandsagents.com> **Amazon Bedrock AgentCore:** <https://docs.aws.amazon.com/bedrock-agentcore/>